

1. Why is GET appropriate for `/api/v1/books`?

GET is appropriate because the purpose of this endpoint is to **retrieve or read information about books** from the server. It does not create, update, or delete data.

In REST APIs:

- GET → Retrieve data
- POST → Create new data
- PUT/PATCH → Update data
- DELETE → Delete data

Therefore, GET `/api/v1/books` means **retrieve the list of books**.

2. What is the purpose of `@RestController`?

`@RestController` tells Spring that the class is a **REST controller** whose methods return data directly in the HTTP response body, usually as JSON, rather than returning an HTML page.

It is essentially a combination of:

```
@Controller
@ResponseBody
```

For example:

```
@RestController
public class BookController {

    @GetMapping("/books")
    public List<Book> getBooks() {
        return books;
    }
}
```

Spring automatically converts the returned Java objects into a suitable response format, commonly JSON.

3. What does `<Books>` mean in `List<Books>`?

`<Books>` specifies the **type of objects that the List can contain**.

For example:

```
List<Books>
```

means that the list contains objects of type `Books`.

If the class is named `Book`, the more common Java convention would be:

```
List<Book>
```

This means a list containing multiple `Book` objects.

4. Why is `books` plural in the URI, and what does `v1` represent?

`books` is plural because the URI represents a **collection of books**, rather than a single book.

For example:

```
GET /api/v1/books
```

means retrieving the collection of books.

A specific book could be accessed using an ID:

```
GET /api/v1/books/10
```

which would normally mean retrieving the book with ID 10.

`v1` means **Version 1** of the API. API versioning allows developers to introduce a new version in the future without necessarily breaking clients that still use the older version.

For example:

```
/api/v1/books  
/api/v2/books
```

5. How does a Java `List<Book>` become JSON without manually writing JSON code?

Spring Boot typically uses **Jackson** to automatically serialize Java objects into JSON.

For example, if the controller returns:

```
List<Book> books
```

Spring passes the returned object to Jackson, which converts it into JSON such as:

```
[
  {
    "id": 1,
    "title": "Java Programming"
  },
  {
    "id": 2,
    "title": "Spring Boot"
  }
]
```

So the developer does not need to manually construct a JSON string.

The process is essentially:

```
List<Book>
  ↓
Spring MVC
  ↓
Jackson
  ↓
JSON
  ↓
HTTP Response
```

6. What would happen if `BookController` were placed outside the package hierarchy scanned by the Spring Boot application?

If `BookController` is placed outside the package hierarchy that Spring Boot scans for components, Spring may **not discover or register the controller as a Spring bean**.

As a result, its endpoints would not be available, and a request to those endpoints would typically result in a **404 Not Found** response.

For example, if the main application class is:

```
com.example.app.Application
```

and the controller is:

```
com.example.app.controller.BookController
```

the controller is normally discovered because it is inside the application's component-scan hierarchy.

However, if it were placed somewhere outside that hierarchy, such as:

```
com.other.controller.BookController
```

Spring would not normally detect it unless component scanning were explicitly configured to include that package.